

llm-jp/llm-jp-3.1-13b like 2 Follow LLM-jp 313

Text Generation Transformers Safetensors English Japanese
llama text-generation-inference License: apache-2.0

Train Deploy Use this model

Model card Files xet Community

Downloads last month
650



Safetensors

Model size 13.7B params Tensor type BF16 Files info

Inference Providers NEW

Text Generation

This model isn't deployed by any Inference Provider.

Ask for provider support

Model tree for llm-jp/llm-jp-3.1-13b

Finetunes 1 model

Collection including llm-jp/llm-jp-3.1-13b

LLM-jp-3.1 Pre-trained Models Collection

Pre-trained models in the LLM-jp-3.1 mod... • 3 items • Updated May 30

llm-jp-3.1-13b

LLM-jp-3.1 is a series of large language models developed by the [Research and Development Center for Large Language Models](#) at the [National Institute of Informatics](#).

Building upon the LLM-jp-3 series, the LLM-jp-3.1 models incorporate mid-training ([instruction pre-training](#)), which significantly enhances their instruction-following capabilities compared to the original LLM-jp-3 models.

This repository provides the **llm-jp-3.1-13b** model. For an overview of the LLM-jp-3.1 models across different parameter sizes, please refer to:

- [LLM-jp-3.1 Pre-trained Models](#)
- [LLM-jp-3.1 Fine-tuned Models](#).

For more details on the training procedures and evaluation results, please refer to [this blog post](#) (in Japanese).

Checkpoints format: Hugging Face Transformers

🔗 Required Libraries and Their Versions

- torch>=2.3.0
- transformers>=4.40.1
- tokenizers>=0.19.1
- accelerate>=0.29.3
- flash-attn>=2.5.8

🔗 Usage

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
tokenizer = AutoTokenizer.from_pretrained("llm-jp/llm-jp-3.1-13b")
model = AutoModelForCausalLM.from_pretrained("llm-jp/llm-jp-3.1-13b", device_map="auto")
text = "自然言語処理とは何か"
tokenized_input = tokenizer.encode(text, add_special_tokens=False, return_tensors="pt")
with torch.no_grad():
    output = model.generate(
        tokenized_input,
        max_new_tokens=100,
        do_sample=True,
        top_p=0.95,
        temperature=0.7,
        repetition_penalty=1.05,
```

```
) [0]  
print(tokenizer.decode(output))
```

Model Details

- Model type:** Transformer-based Language Model
- Architectures:**

Dense model:

Params	Layers	Hidden size	Heads	Context length	Embedding parameters	Non-embedding parameters
1.8b	24	2048	16	4096	407,498,752	1,459,718,144
13b	40	5120	40	4096	1,018,746,880	12,688,184,320

MoE model:

Params	Layers	Hidden size	Heads	Routed Experts	Activated Experts	Context length	Embedding parameters	Non-embedding parameters	Act para
8x13b	40	5120	40	8	2	4096	1,018,746,880	72,144,081,920	22,200

Tokenizer

The tokenizer of this model is based on [huggingface/tokenizers](#) Unigram byte-fallback model. The vocabulary entries were converted from [llm-jp-tokenizer v3.0](#). Please refer to [README.md](#) of [llm-jp-tokenizer](#) for details on the vocabulary construction procedure (the pure SentencePiece training does not reproduce our vocabulary).

[🔗 Datasets](#)

[🔗 Pre-training](#)

The models have been pre-trained using a blend of the following datasets.

Language	Dataset	Tokens
Japanese	Wikipedia	2.6B
	Common Crawl	762.8B
	WARP/PDF	237.3B
	WARP/HTML	2.7B
	Kaken	1.8B
English	Wikipedia	4.7B
	Dolma/CC-head	608.5B
	Dolma/C4	181.6B
	Dolma/Reddit	83.1B
	Dolma/PeS2o	62.9B
	Dolma/Gutenberg	5.5B
	Dolma/Wiki	3.9B
Code	The Stack	114.1B
Chinese	Wikipedia	0.8B
Korean	Wikipedia	0.3B

[🔗 Mid-training](#)

In the LLM-jp-3.1 series, we performed continuous pre-training based on [Instruction Pre-Training](#). Instruction Pre-Training enhances a model's ability to follow instructions by continuing pre-training on a large collection of instruction–response pairs. We prepared approximately 90B tokens of instruction–response data and mixed it with our pre-training datasets, conducting continuous pre-training on a total of 400B tokens. Each model was initialized from existing checkpoints ([llm-jp/llm-jp-3-1.8b](#), [llm-jp/llm-jp-3-13b](#), and [llm-jp/llm-jp-3-8x13b](#)) and underwent continuous instruction pre-training. Since the LLM-jp-3 series was originally pre-trained on 2.1T tokens, the total pre-training token count amounts to 2.5T tokens.

Details of this training process will be released in a forthcoming paper. The instruction–response dataset used for this training will also be made publicly available.

[🔗](#) Post-training

We have fine-tuned the pre-trained checkpoint with supervised fine-tuning and further aligned it with Direct Preference Optimization.

[🔗](#) Supervised Fine-tuning

The datasets used for supervised fine-tuning are as follows:

Language	Dataset	Description
Japanese	ichikara-instruction-004-002	A manually constructed instruction dataset.
	AnswerCarefully (ver2.0)	A manually constructed instruction dataset focusing on LLMs' safety.
	ichikara-instruction-format	A small subset of the ichikara-instruction dataset, edited with some constraints on the output format.
	AutoMultiTurnByCalm3-22B	A synthetic instruction dataset.
	random-to-fixed-multiturn-Calm3	A synthetic instruction dataset.
	wizardlm8x22b-logical-math-coding-sft-ja	A synthetic instruction dataset.

Language	Dataset	Description
	magpie-sft-v1.0	A synthetic instruction dataset we created.
	jaster v1.4.1	-
	extraction-wiki-ja	A synthetic instruction dataset we created.
English	Daring-Anteater	-
Japanese & English	Synthetic-JP-EN-Coding-Dataset	A synthetic instruction dataset.

[🔗](#) **Direct Preference Optimization**

For Direct Preference Optimization (DPO), we adopted rejection sampling. Prompts were sampled from the dataset used in SFT, and multiple responses were generated for each prompt. These responses were then scored (by [Qwen/Qwen2.5-32B-Instruct](#)), and DPO was performed by treating high-scoring responses as positive examples and low-scoring responses as negative examples.

We conducted DPO in two stages. In the second stage, we additionally used [ac-self-inst](#), a Japanese preference dataset focused on safety.

[🔗](#) **Evaluation**

[🔗](#) **MT Bench (Japanese and English)**

We evaluated the models using `gpt-4o-2024-08-06`. The scores represent the average values obtained from three rounds of inference and evaluation. For more details, please refer to the [codes](#).

Model Name	JA	EN
gpt-35-turbo-1106	6.48	7.56
gpt-4-0613	7.29	7.72
gpt-4o-2024-08-06	8.10	8.38

Model Name	JA	EN
<u>sbintuitions/sarashina2.2-1b-instruct-v0.1</u>	5.30	5.66
<u>sbintuitions/sarashina2.2-3b-instruct-v0.1</u>	7.07	6.96
<u>Rakuten/RakutenAI-2.0-8x7B-instruct</u>	6.68	6.33
<u>cyberagent/calm3-22b-chat</u>	6.86	6.77
<u>Qwen/Qwen2.5-14B-Instruct</u>	7.07	7.99
<u>Qwen/Qwen2.5-32B-Instruct</u>	7.64	8.27
<u>Qwen/Qwen3-1.7B</u>	5.46	6.95
<u>Qwen/Qwen3-14B</u>	8.00	8.30
<u>Qwen/Qwen3-32B</u>	8.36	8.33
<u>tokyotech-llm/Llama-3.3-Swallow-70B-Instruct-v0.4</u>	7.64	8.02
<u>stockmark/Stockmark-2-100B-Instruct-beta</u>	7.42	7.17
<u>llm-jp-3-1.8b-instruct3</u>	4.64	4.09
<u>llm-jp-3-13b-instruct3</u>	6.21	6.13
<u>llm-jp-3-8x13b-instruct3</u>	6.60	6.49
<u>llm-jp-3.1-1.8b-instruct4</u>	6.30	5.70
<u>llm-jp-3.1-13b-instruct4</u>	7.37	7.01
<u>llm-jp-3.1-8x13b-instruct4</u>	7.50	7.05

[AnswerCarefully-Eval](#)

[AnswerCarefully-Eval](#) assesses the safety of Japanese language model outputs using the LLM-as-a-Judge approach, based on the test set from [llm-jp/AnswerCarefully](#). We evaluated the models using `gpt-4o-2024-08-06`. The scores represent the average values obtained from three rounds of inference and evaluation. For more details, please refer to the [codes](#).

Model name	Score	Acceptance rate (% , ↑)	Violation rate (% , ↓)
gpt-35-turbo-1106	3.98	71.7	12.6
gpt-4-0613	4.06	72.3	13.2
gpt-4o-2024-08-06	4.09	72.7	12.5
<u>llm-jp-3-1.8b-instruct3</u>	4.03	75.9	12.2
<u>llm-jp-3-13b-instruct3</u>	4.37	88.4	6.5
<u>llm-jp-3-8x13b-instruct3</u>	4.48	91.6	4.3
<u>llm-jp-3.1-1.8b-instruct4</u>	3.66	64.7	24.3
<u>llm-jp-3.1-13b-instruct4</u>	4.17	82.4	12.2
<u>llm-jp-3.1-8x13b-instruct4</u>	4.26	83.1	11.6

[🔗 Risks and Limitations](#)

The models released here are in the early stages of our research and development and have not been tuned to ensure outputs align with human intent and safety considerations.

[🔗 Send Questions to](#)

llm-jp(at)nii.ac.jp

[🔗 License](#)

[Apache License, Version 2.0](#)

[🔗 Model Card Authors](#)

The names are listed in alphabetical order.

Hirokazu Kiyomaru and Takashi Kodama.